

■ A.1 Defining Mathematical Functions

■ A.1.1 Some Combinatorial Functions

Probably the simplest kinds of *Mathematica* packages are ones that define simple mathematical functions in terms of mathematical functions that are built into *Mathematica*.

As a first example, we consider the definition of Catalan numbers. These arise, for example, in counting the number of possible tree structures of a certain size. Here is the definition of Catalan numbers given in the package `CombinatorialFunctions.m`. Following standard *Mathematica* conventions, the function that gives Catalan numbers is named `CatalanNumber`.

```
CatalanNumber::usage = "CatalanNumber[n] gives the nth Catalan number."

CatalanNumber[n_Integer] := Binomial[2n, n]/(n + 1)
```

The definition of Catalan numbers.

Notice that in addition to the actual *Mathematica* definition for `CatalanNumber`, the package gives a `usage` message. This is retrieved when you ask for information using `?` in *Mathematica*.

```
Subfactorial::usage =
  "Subfactorial[n] gives the number of permutations of n objects
  which leave no object fixed."

Subfactorial[n_Integer?Positive] := n! Block[{k}, Sum[(-1)^k/k!, {k, 0, n}]]
```

The subfactorial function.

The `Subfactorial` function is defined using `Sum`. Notice that the dummy variable `k` in the sum is specified as being local to a block. This ensures that the variable used in this definition will not get confused with other variables named `k`.

```
Fibonacci::usage = "Fibonacci[n] gives the nth Fibonacci number."

Fibonacci[n_Integer?Positive] := Fibonacci[n] = Fibonacci[n-1] + Fibonacci[n-2]

Fibonacci[0] = Fibonacci[1] = 1
```

The Fibonacci numbers.

Fibonacci numbers are defined here using recursion. Notice that the argument

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

to `Fibonacci` is specified to be positive. It is important to put in constraints like this to make sure that recursive procedures terminate properly. Notice also that `Fibonacci` stores each value that it generates. This is a common method for optimizing the evaluation of recursive functions.

This reads in the package
`CombinatorialFunctions.m`. This package
may or not exist on your particular
computer system.

```
In[1]:= <<CombinatorialFunctions.m
```

Typing `?` gives you the usage message for a
particular object.

```
In[2]:= ?CatalanNumber
CatalanNumber[n] gives the nth Catalan number.
```

Here is the 20th Catalan number.

```
In[2]:= CatalanNumber[20]
Out[2]= 6564120420
```

The definitions give above are in fact fragments from *Mathematica* packages. A complete *Mathematica* package contains in addition context specifications. Some packages also write `protect` the objects they define.

```
BeginPackage["Hofstadter`"]

Hofstadter::usage = "Hofstadter[n] gives Hofstadter's function."

(* Recursive definition of the Hofstadter function *)

Hofstadter[n_Integer?Positive] :=
    Hofstadter[n] =
        Hofstadter[n - Hofstadter[n-1]] + Hofstadter[n - Hofstadter[n-2]]

Hofstadter[1] = Hofstadter[2] = 1

EndPackage[ ]
```

A complete package for the Hofstadter function.